

cc-2 1) Requirements - Requirement Development Methodology

Requirements Engineering:

Requirements engineering is the process of gathering, analyzing, documenting, and managing project needs to ensure a successful outcome. It bridges the gap between customer expectations and system development.

Types of Requirements

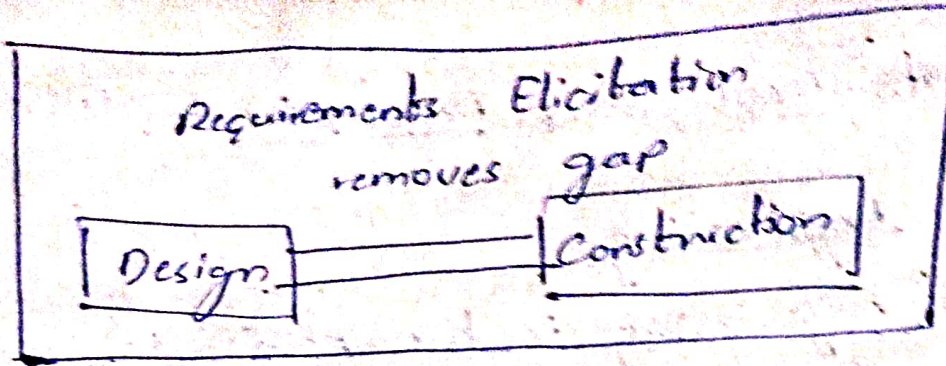
- Project Requirements - Guidelines for a project
- Software Requirements - functional and non-functional system needs
- Personal Requirements - Individual needs for a goal
- Documentation Requirements - formatting and structure rules

Requirements Elicitation

The process of identifying and documenting stakeholders' needs for system development. It ensures alignment with user expectations and business goals.

Examples: student profile Elicitation

- know the student profile structure
- use a template for consistency.



Types of Templates

- 1) Document Templates
purpose: standardized layouts for documents
Exp: Resumes, cover letters, reports.
- 2) Software Development Templates
P: Reusable code structures for efficient development
E: HTML templates, C++ class templates, Django templates
- 3) Email Templates
P: Prewritten formats for repetitive communication
E: Customer service responses, newsletters
- 4) Design Template
P: Standardized designs for graphics and presentations.
E: Canva posters, Power Point slides.
- 5) Database Templates
P: Predelinead database structures for specific use cases
E: CRM systems, inventory management templates

Benefits of Using Templates:

- Saves Time
- Ensures Consistency
- Reduces Errors
- Professional & Polished Output

Q) Prepare student profile template.

student Profile

XYZ Private Ltd, Pin-500086

Employee Profile

Emp

Employee Name

student Profile

1. Roll No:

2. Name:

3. Department:

4. Year:

5. Branch:

6. DOB:

7. Mobile:

8. Email:

231003005
231003015
231003026

1.2) Specifying Requirements

Specifying requirements involves documenting and validating gathered requirements to ensure they align with stakeholder need. This phase is crucial for accurate software design, development and testing.

Verification vs. Validation

Aspects	Verification	Validation
Definition	Ensures the software is built correctly as per specifications	Ensures the software meets user needs and intended purpose
Focus	"Are we building the product right?"	"Are we building the right product?"
Methods	Reviews, inspections, walkthroughs, testing phases	User testing, acceptance testing, usability checks
Stage	Conducted during development	Conducted after development (before release)
Example	Checking if a login function works as per design	Checking if the login function meets user expectations

1.3) Eliciting Accurate Requirement

Gathering and documenting stakeholder needs is crucial for software success.

Steps to Elicit Requirements:

1. Identify stakeholders - End-users, clients, developers, testers

2. Prepare for Elicitation - Understand project goals, scope, and existing data.
3. Choose Techniques - Interviews, workshops, surveys, prototyping, observation.
4. Conduct Sessions - Gather business goals, functional & non-functional needs.
5. Validate & Prioritize - Use methods like Moscow and value vs. effort analysis.
6. Document Requirements - Include functional, non-functional, UI, and domain-specific needs.

Functional vs. Non-functional Requirements

1. Functional Requirements

Define the system's expected behavior and functionalities. They describe what the system should do.

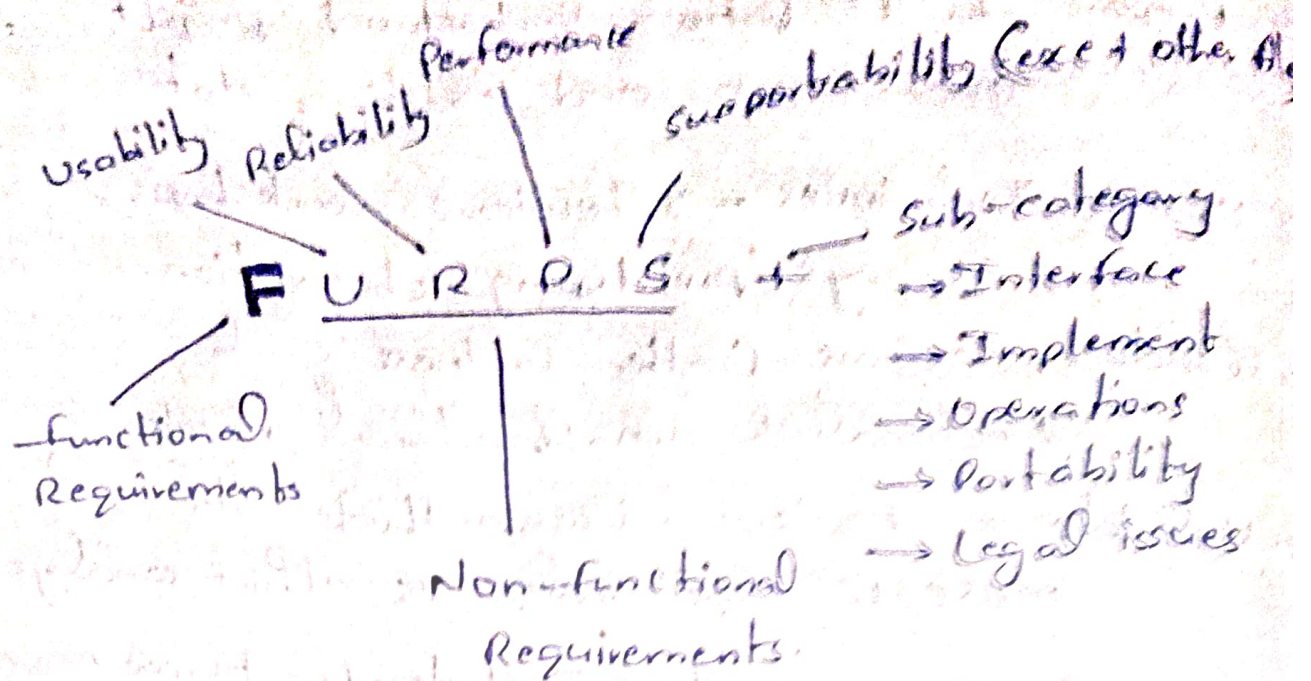
Example: Payment processing, generating reports.

2. Non-functional Requirements

Specify system constraints and quality attributes. They define how the system should perform.

Example: & Types:

- Performance
- Scalability
- Usability
- Security
- Availability
- Compliance



7) Manage changes - Use requirement management tools for updates and traceability.

challenges:

- Ambiguity - Unclear stakeholder inputs
- Conflicting Needs - Different expectations from groups.
- changing Requirements - Evolving project demands.
- Incomplete Information - Missing or overlooked details.

Currency format Examples:

- USA - \$450,000.00
- Germany - €450.000.00
- India - ₹4,50,000.00

1.4) Documenting Business Requirements

- 1) Understand Business Needs
 - Gather stakeholder insights via meeting, interviews, or workshops
 - Define goals & objectives (e.g. "Improve inventory efficiency by 25% in 6 months").

- 2) Business Requirements Document (BRD) structure
 - Introduction: Project background, purpose, stakeholders.

- Business objectives: key goals (e.g.: "Enhance customer experience with a self-service portal").

- scope: Define what's included/excluded (e.g.: "In-scope: Portal development, out-of-scope: Hardware")

- Requirements:
 - functional: system actions (What to do)
 - Non-functional: Quality constraints (How to do)

- Assumptions & Constraints: Expected conditions & limitations (e.g.: "Budget: \$50,000")

- Stakeholders & Roles: Define responsibilities for approvals & reviews.

- Glossary: clarify key terms.

- 3) Requirement Gathering Techniques

- Interviews, Workshops, Surveys, Observation, Document Analysis.

- 4) Writing Effective Requirements

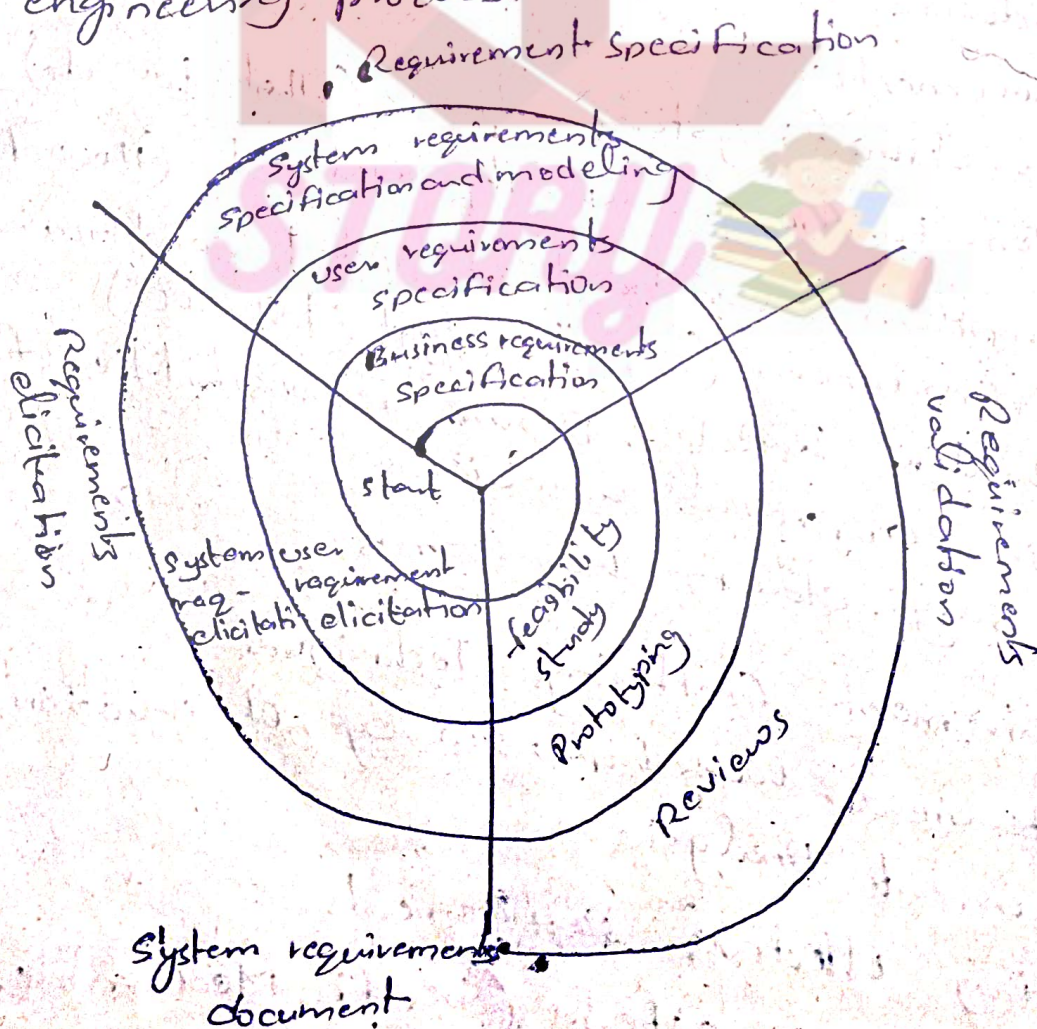
- Use SMART criteria (Specific, Measurable, Achievable, Relevant, Time-bound).

- 5) Visual Aids for clarity
→ flowcharts, Use case Diagrams, Wireframes
- 6) Validation & sign-off
→ Review, validate with stakeholders, and get approval.
- 7) Main updates
→ Keep BRD version-controlled & updated with changes

Examples:

- functional Requirement
- Non-functional Requirement

1.5) A spiral view of the requirements engineering process.



Defining User Requirements

User requirements outline what users need from a system, ensuring software meets expectations and business goals.

Types of User Requirements:

- functional : Define specific system features
- Non-functional : Define system qualities

Methods to Define User Requirements:

- User Stories : "As a user, I want to track progress to monitor improvement".
- Use Cases : step-by-step user interactions.
- Surveys & Interviews : Gather insights.
- Prototypes & wireframes : Visual representations.

Best Practices

- Be specific & clear.
- Make it measurable & Testable
- Prioritize Requirements
- Validate with stakeholders
- Maintain a Living Document

Common Pitfall to Avoid (Disadvantages)

- Ambiguity
- Incomplete Requirements
- Ignoring Non-functional Needs
- Scope Creep
- Lack of validation

User Stories Examples:

- Banking
- Loan Management

What is a persona?

A persona is a fictional user profile based on research, helping teams design user-centered solutions.

1.6) Validating Requirements

Requirement validation ensures correctness, completeness, and alignment with user needs, preventing costly rework.

Key Validation checks:

1. Validity - Ensure requirements reflect real user needs.
2. Consistency - Avoid conflicts
3. Completeness - Cover all necessary functions
4. Realism - Ensure feasibility within budget
5. Verifiability - Write testable requirements

Validation Techniques:

- Requirements Review - systematic analysis to identify errors.
- Prototyping - Create models for user feedback
- Test Case Generation - Develop tests to validate feasibility.
- Requirement Traceability Matrix (RTM) - Map requirements to design and tests.
- Stakeholder Feedback & Workshops - Ensure alignment with expectations.

→ formal Inspections & Audit + Verify compliance with standards.

1.7) Achieving Requirements Traceability

Requirements traceability ensures all requirements are tracked from inception to deployment.

How to Achieve it?

1) Requirement Traceability Matrix (RTM) :-

Maps requirements to design, implementation, and tests.

Requirement ID	Requirement Description	Design Reference	Implementation Code	Test Case ID	Status
REQ-01	The chatbot should provide mental health tips based on user mood.	UI Design Doc	chatbot.py	TC-101	Passed
REQ-02	Users should be able to reset passwords via email.	Security Design Doc	auth.py	TC-102	In progress

2) Use Tracking Tools - JIRA, IBM DOORS, Azure DevOps, Trello, Asana.

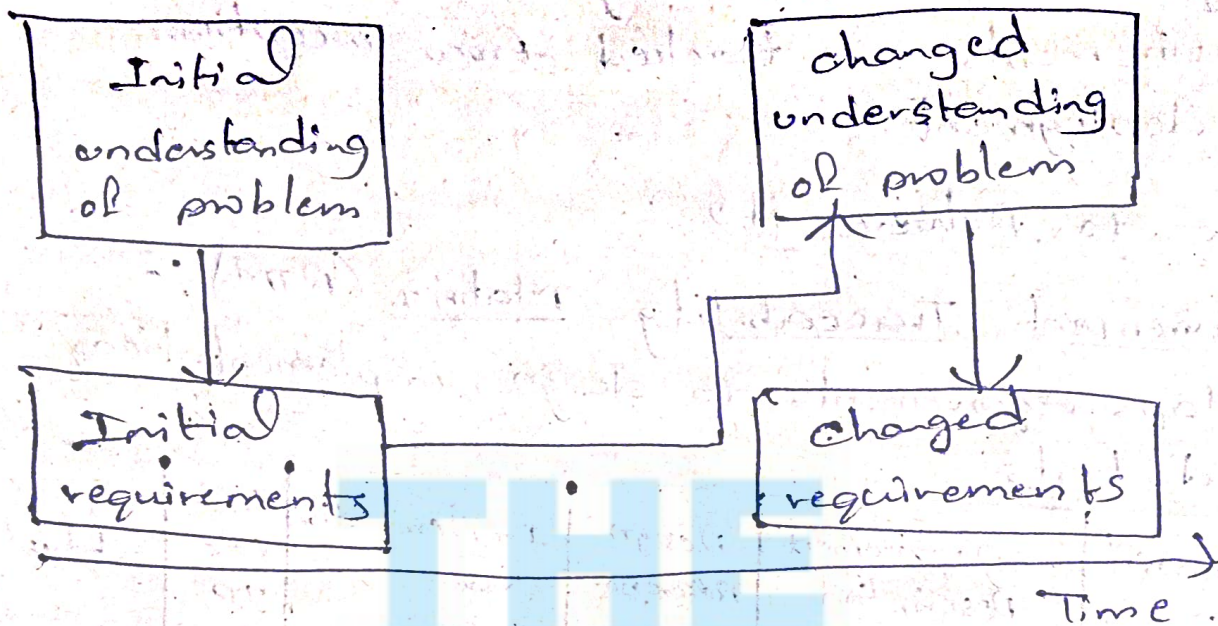
3) Unique Requirement IDs - Assign IDs (e.g., REQ-001) for easy tracking.

4) Change Management - Track and update changes in linked artifacts.

5) Regular Reviews - Ensure completeness and alignment with project goals.

1.8) Managing changing Requirements

Requirements evolve due to shifting business, technical, and stakeholder needs.



Why do requirements change?

- Evolving Business & Tech - New hardware, system integrations, regulatory updates
- Customer vs. User needs - Budget constraints vs. actual user demands
- Diverse stakeholders - Conflicting priorities may require re-prioritization.

Example: Construction Blueprints.

Blueprints change due to design modification, site conditions, or regulations. Old versions are archived for compliance, legal protection, and reference.

1.9) Reviews

Requirement reviews are essential in software and system development to ensure that requirements are complete, consistent, unambiguous, and aligned with stakeholders needs before implementation begins. These reviews help identify errors, ambiguities, and missing elements early, reducing costly changes later in development.

2.0) walkthroughs (informal)

A walkthrough is a structured yet informal review process where the author of the requirements document explains the content to a group of stakeholders to gather feedback and ensure correctness, clarity, and completeness.

Example: Scenario

Walkthrough Discussion

→ BA

→ Developer

→ Tester

→ Project Manager

2.1) Inspections (formal)

A requirement inspection is a formal, structured review process where a trained team examines a requirements document to identify errors, ambiguities, inconsistencies, and missing details moving forward with design and development.

2.2) SRS vs User stories

features	SRS ^{Software requirement specification}	User stories
Definition	Detailed system requirements document	Brief user-centric feature descriptions
Purpose	Ensures clarity before development	Guides Agile teams in prioritizing features
Scope	Comprehensive, static	High-level, flexible
Format	Formal, structured	Simple: "As a [user] I want [feature] so that [reason]"
Audience	Developers, testers, stakeholders	Agile teams, stakeholders
Flexibility	Rigid, requires approval for changes	Adaptive, evolves with development
Approach	Waterfall (fixed requirements)	Agile (Incremental)
Example	Rating & Comment	Feedback to rate self experience

2.3) Agile Overview

Agile is fast, iterative, and flexible software development approach that emphasizes incremental delivery, continuous feedback, and adaptability to changing requirements. It focus on customer satisfaction and cross-functional collaboration.

Agile Process :

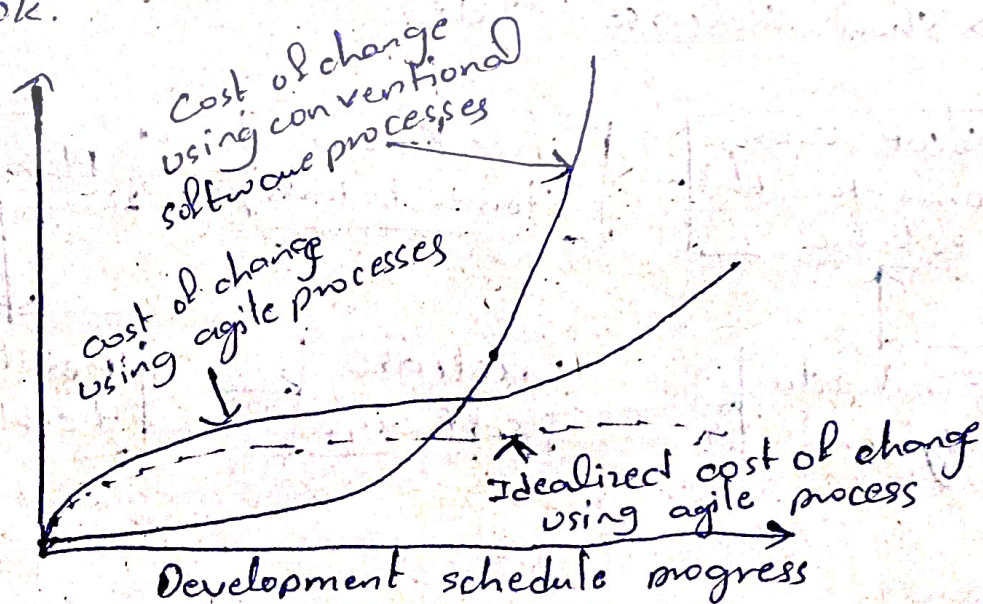
- Brainstorm - Gather requirements
- Design - Create prototype & documents
- Develop - Iterative coding & feedback
- QA - Testing & feedback (Quality Assurance)
- Deploy - Production & support

Agile Modeling.

Agile Modeling (AM) is a lightweight and flexible approach to software design, focusing on just enough documentation to support development.

Agile software development process (Iterative step & back to step 1)

- 1) Set requirements
- 2) Design
- 3) Development / Implement / coding
- 4) Testing
- 5) Deployment
- 6) Review
- 7) Delivery
- 8) feedback.



Benefits

- faster development
- Better collaboration
- flexible & adaptable
- Improved communication
- Reduced waste

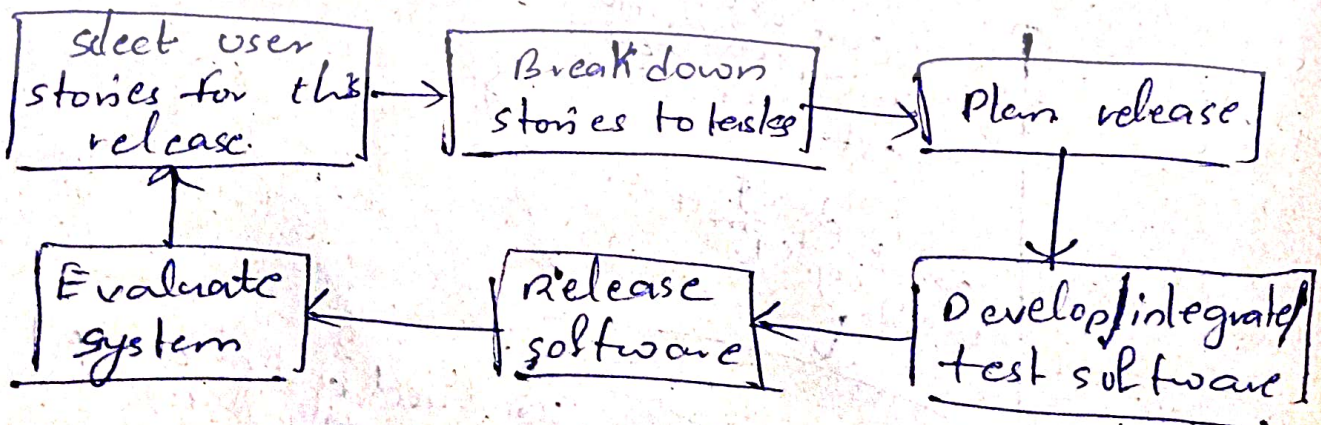
When to Use?

- when requirements evolve
- when frequent feedback is needed
- when collaboration & visual models aid development

2.4) Extreme Programming (XP)

XP is an Agile methodology introduced by Kent Beck that pushes best practices to the extreme. It emphasizes:

- User stories
- Pair Programming
- Test-Driven Development (TDD)
- frequent integration
- short Release cycles.



Heavyweight vs. Lightweight Methodologies

Heavyweight (Traditional)

- Extensive planning & documentation
- Rigid, sequential processes (e.g. waterfall, V-Model)
- Best for large-scale projects with fixed requirements

Lightweight (Agile)

- Iterative, flexible, and customer-focused
- Minimal documentation & continuous feedback (e.g. scrum, XP, Kanban)
- Best for startups & fast-changing projects

Wireframe:

A wireframe is a basic, low-fidelity layout of a webpage or app, focusing on structure, content placement, and UX without design elements.

← CO-2 End →